

SQL-инъекция

Описание проблемы

Это уязвимость, позволяющая атакующему модифицировать запросы, которые приложение делает к своей базе данных.

Модификация возможна, если механизм формирования запросов допускает выход из предполагаемого разработчиком контекста.

Общее правило: если вы используете конкатенацию или форматные строки при формировании запроса к базе данных, то код нужно переписать с использованием подготовленных выражений или платформенного шаблонизатора (см. раздел "[Как исправить?](#)").

Пример небезопасного кода

```
1 username = params.Get('username')
2 SqlQuery(f"SELECT fio FROM users WHERE name = '{username}'")
```

Предполагая, что username мы получаем от пользователя, легко выйти из контекста этого параметра, передав вместо этого параметра

```
' AND 1=0 UNION SELECT version() --
```

что приведет к выполнению такого запроса:

```
SELECT fio FROM users WHERE name='' AND 1=0 UNION SELECT version() -- '
```

То есть вместо ФИО мы получим версию PostgreSQL - в данном примере, а в итоге - любые данные из базы.

Последствия эксплуатации

С технической стороны SQL-инъекция позволяет выполнить запрос к базе данных сервиса.

Это позволяет получить данные из базы или модифицировать их.

А также в некоторых случаях:

- загрузить или модифицировать файлы на удаленной машине (copy from/to);
- выполнить произвольные команды системы (copy from/to program);
- выполнить запрос к другой базе данных (с помощью расширения dblink);
- вызвать отказ в обслуживании (generate_series).

Как исправить?

Подготовленные выражения

Механизм подготовленных выражений (prepared statements) заключается в том, что значения параметров SQL-запроса передаются базе данных отдельно от самого запроса.

Это позволяет базе отличить код от данных, а потому, какие бы значения параметров ни были переданы, это не позволит повлиять на сам запрос.

Основное ограничение такого подхода - параметризовать можно только значения полей, передать в качестве параметра имя идентификатора (например, название таблицы или столбца) не выйдет.

Также при большом количестве параметров использовать подготовленные выражения в чистом виде становится затруднительно, т.к. плейсхолдеры связываются с параметрами просто по их порядковому номеру. Эту проблему решает [платформенный шаблонизатор](#).

Однако, если приведенные недостатки не смущают, данный подход нужно считать предпочтительным.

Для формирования запросов к базе данных с использованием подготовленных выражений в платформе реализовано семейство функций `SqlQuery`.

API C++:

- [SqlQuery](#)
- [SqlQueryScalar](#)

API Python:

- [SqlQuery](#)
- [SqlQueryRecord](#)
- [SqlQueryRow](#)
- [SqlQueryScalar](#)

Платформенный механизм формирования запросов к базе данных с использованием подготовленных выражений описан в данной [статье](#).

Примеры

Обрамлять кавычками плейсхолдер (конструкцию `$1`) не нужно:

```
SqlQuery("SELECT * FROM users WHERE id = $1", user_id)
```

Пример для C++:

```
RecordSet rs = SqlQuery( L"SELECT * FROM my_table WHERE id=$1 AND start_date  
> $2 AND end_date < $3", id, start_date, end_date );
```

Если нужно передать несколько параметров, нужно использовать несколько плейсхолдеров. При этом можно дублировать

```
SqlQuery("INSERT INTO test_table VALUES (NULL, $1, $2, false, $1)", foo, bar)
```

Таким образом можно передавать массивы:

```
SqlQuery("DELETE FROM Token WHERE token = ANY($1)", tokens_list)
```

Платформенный шаблонизатор

Платформенный шаблонизатор позволяет формировать более сложные запросы к базе, чем подготовленные выражения. В частности, он позволяет:

- использовать именованные параметры;
- формировать запросы с необязательными частями;
- удобно хранить шаблоны в отдельных файлах
- кэшировать разобранный шаблон.

После разбора шаблонов получаются [подготовленные выражения](#), поэтому остается их недостаток: плейсхолдеры можно использовать только для литералов, но не для идентификаторов.

При формировании запросов по шаблону эта проблема решается следующим образом: идентификаторы приходится экранировать.

Пример:

```
1 QueryByTemplate(  
2     Template("SELECT * FROM !table WHERE user_id = !user_id"),
```

3 | table=SqlIdentifier(table), user_id=user_id)

В приведенном примере вместо !table в запрос подставится не плейсхолдер, а результат выполнения функции [SqlIdentifier](#), т.е. имя таблицы с экранированными двойными кавычками.

Для формирования запросов к базе данных по шаблону реализованы функции:

API C++:

- [sbis::dbi::Template](#)
- [sbis::dbi::TemplateExecutor](#)
- [sbis::dbi::QueryByTemplate](#)

API Python:

- [sbis.Template](#)
- [sbis.TemplateExecutor](#)
- [sbis.QueryByTemplate](#)
- [sbis.QueryRecordByTemplate](#)
- [sbis.QueryRowByTemplate](#)
- [sbis.QueryScalarByTemplate](#)

Как использовать платформенный шаблонизатор подробно и с примерами описано в [статье](#).

Примеры

- Python

```
1  tpl = Template("""
2      SELECT * FROM "Клиент"
3      LEFT JOIN
4          "Договор"
5          ON "@Клиент" = "Договор"."КлиентСБИС"
6          AND "@Договор" {% = contract %}
7      WHERE
8          "@Клиент" = !account
9      """)
10
11  QueryByTemplate(tpl, contract=None, account=1234)
```

- C++

```
1  static dbi::Template const tpl( L"insert into !table values(!id,
   !name)"_sv );
2
3  dbi::TemplateExecutor tpl_exec( tpl );
4
5  tpl_exec.AddParam( L"table"_sv, dbi::SqlIdentifier( L"таблица"_sv )
   );
6
7  tpl_exec.AddParam( L"id"_sv, 1 );
8  tpl_exec.AddParam( L"name"_sv, L"название" );
9
10 auto st = db.CreateStatement();
11
12 st->Exec( tpl_exec.SetupQuery( *st ) );
```

Как делать не нужно

Писать самодельный шаблонизатор

Если вы считаете, что

- ваш шаблонизатор будет лучше платформенного,

- в платформенном нет нужного функционала,
- то более предпочтительным вариантом будет доработать платформенный шаблонизатор.

Экранировать спецсимволы вручную

не следует, т.к. это может привести к ошибкам. Тем более, что для этих целей уже есть

- метод [SqlIdentifier](#) - для идентификаторов, т.е. название таблицы, колонок и т.д.;
- метод [SqlLiteral](#) - для литералов, т.е. значений параметров.

Важно не перепутать эти методы, нельзя использовать `SqlLiteral` для экранирования идентификаторов и наоборот.

Использование платформенных методов позволит сделать код понятнее для тех, кто хочет в нем разобраться.

Санитайзить данные вручную

В частности:

- убирать из текстовых данных спецсимволы,
- приводить данные к "безопасному" типу, например, числовому или `uuid`.

Эти способы работают, если их реализовать правильно, однако ошибиться здесь гораздо проще, чем при использовании платформенных классов.

В случае приведения типа трудно ошибиться, однако его можно легко убрать при рефакторинге, забыв, зачем оно было добавлено.

Также нестандартные способы формирования SQL-запросов затрудняют анализ кода, в том числе - автоматизированными средствами (SAST).

Полагаться на проверку типов платформой

Если вы указали тип параметра в сигнатуре метода, то платформа попытается привести тип указанному. В случае, если это сделать не удастся, возникнет ошибка и метод не будет вызван.

Таким образом, если указать в качестве типа, к примеру, "Число целое", то данный параметр можно будет считать безопасным, ведь передать произвольную строку вместо числа для эксплуатации SQL-инъекции не получится.

Однако полагаться на этот механизм для защиты от инъекций не нужно, поскольку сигнатуры методов могут быть изменены со временем и разработчик, изменяющий сигнатуру, может даже не вспомнить, что тип того или иного параметра должен быть "безопасным" с точки зрения SQL-инъекций.

Также функция, содержащая запрос к базе, может использоваться в нескольких местах - в этом случае придется следить уже за сигнатурами нескольких методов.